

Section 1

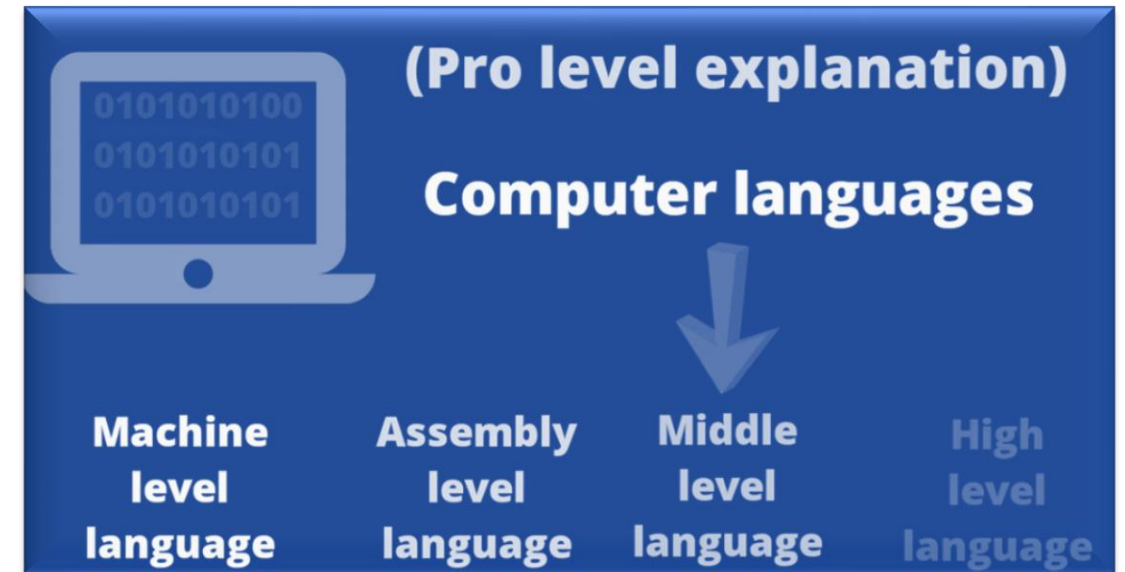
Systems Programming ECE3502

Translators

- A translator or programming language processor that converts a computer program from language to another.

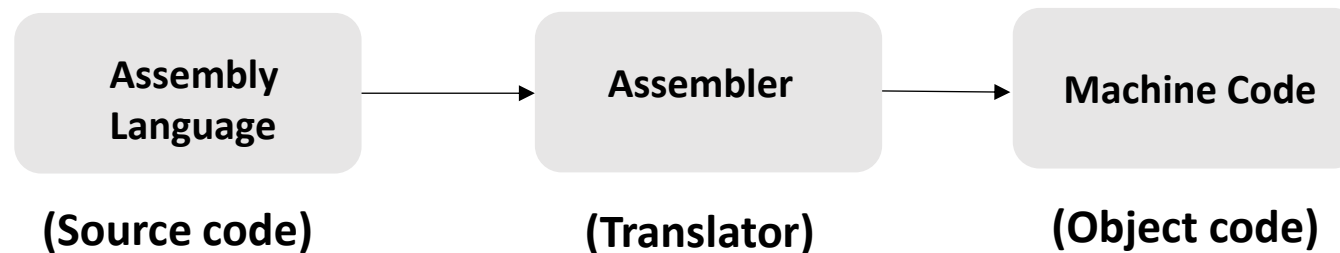
(convert the programming language into a machine language)

- It takes a program written in source code and converts into machine code.
- It discovers and identifies the error during translation.



Types of Translators

Assembler: the assembler is used to translate the program written in assembly language into machine level language.



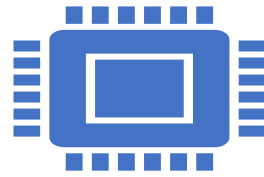
Compilers: the compiler will read the complete source program written in high level language, if the code is error free then it will convert into machine level language.

Interpreter: just like compiler, it convert one at a time and reports errors detect once, while doing conversion (faster than a compiler).

SIC Machine

- This machine has been designed to illustrate the most commonly encountered hardware features and concepts.
- SIC (Simplified Instruction Computer) comes in two versions:
 1. Standard model
 2. Extra Expensive/ Extra Equipment/ XE version

SIC machine architecture



Memory

it consists of 8-bit bytes

3 consecutive byte from a word(24 bits)

Total of 32768 (2^{15}) bytes in the computer memory



Data Formats

Integers are stored as 24-bit binary numbers

2'S complement representation is used for negative values

Characters are stored using their 8-bit ASCII codes

No floating-point hardware on the standard version of sic

Registers

There are 5 registers

Mnemonic	Number	Special Use
A	0	Accumulator
X	1	Index Register
L	2	Linkage Register
PC	8	Program Counter
SW	9	Status Word

Instructions Set

Category	Mnemonic
Load and Store	LDA,LDX,STA,STX
Arithmetic Operation	ADD, SUB, MUL, DIV
Compare	COMP
Conditional Code	JLT, JEQ, JGT
Conditional Code Subroutine Instructions	JSUB
	RSUB

Instruction Set Sheet

A stands for Accumulator

M stands for Memory

CC stands for Condition Code

PC stands for Program Counter

RMB stands for Right Most Byte

L stands for Linkage Register

MNEMONIC	OPERAND	OPCODE	EXPLANATION
ADD	M	18	$A = A + M$
AND	M	40	$A = A \text{ AND } M$
COMP	M	28	compares A and M
DIV	M	24	$A = A / M$
J	M	3C	$PC = M$
JEQ	M	30	if CC set to =, $PC = M$
JGT	M	34	if CC set to >, $PC = M$
JLT	M	38	if CC set to <, $PC = M$
JSUB	M	48	$L = PC ; PC = M$
LDA	M	00	$A = M$

Instruction Set Sheet

A stands for Accumulator

M stands for Memory

CC stands for Condition Code

PC stands for Program Counter

RMB stands for Right Most Byte

L stands for Linkage Register

LDCH	M	50	$A[RMB] = M[RMB]$
LDL	M	08	$L = M$
LDX	M	04	$X = M$
MUL	M	20	$A = A * M$
OR	M	44	$A = A \text{ OR } M$
RD	M	D8	$A[RMB] = \text{data specified by } M[RMB]$
RSUB		4C	$PC = L$
STA	M	0C	$M = A$
STCH	M	54	$M[RMB] = A[RMB]$
STL	M	14	$M = L$
STSW	M	E8	$M = SW$
STX	M	10	$M = X$
SUB	M	1C	$A = A - M$

Assembler Directives

Basic assembler directives

- **START:** Specify name and starting address for the program
- **END:** Indicate the end of the source program, and (optionally) the first executable instruction in the program.
- **BYTE:** Generate character or hexadecimal constant, occupying as many bytes as needed to represent the constant.
- **WORD:** Generate one-word integer constant
- **RESB:** Reserve the indicated number of bytes for a data area
- **RESW:** Reserve the indicated number of words for a data area

Trace the following
assembly code for SIC

Program 1:

	LDA	BETA
	DIV	GAMMA
	STA	ALPHA
	:	
	:	
ALPHA	RESW	1
BETA	WORD	10
GAMMA	WORD	2

Trace the following
assembly code for SIC

Program 2:

```
LDA    FIVE
STA    ALPHA
LDCH   CHARZ
STCH   C1
```

:

:

ALPHA	RESW	1
FIVE	WORD	5
CHARZ	BYTE	C'5'
C1	RESB	1

Trace the following
assembly code for SIC

Program 3:

	LDX	ZERO
MOVECH	LDCH	STR1,X
	STCH	STR2,X
	TIX	ELEVEN
	JLT	MOVECH
	:	
	:	
STR1	BYTE	C'TEST STRING'
STR2	RESB	11
ZERO	WORD	0
ELEVEN	WORD	11

Trace the following
assembly code for SIC

Program 4:

LDA	ALPHA
ADD	INCR
SUB	ONE
STA	BETA
LDA	GAMMA
ADD	INCR
SUB	ONE
STA	DELTA

:

:

ONE	WORD	1
ALPHA	WORD	10
INCR	WORD	2
BETA	RESW	1
GAMMA	WORD	15
DELTA	RESW	1

Sheet 1

Question 4

Write a sequence of instructions for SIC to set ALPHA equal to the integer portion of $BETA \div GAMMA$.

LDA BETA

DIV GAMMA

STA ALPHA

:

:

ALPHA	RESW	1
BETA	WORD	8
GAMMA	WORD	4

Sheet 1

Question 2

Write SIC instructions
to swap the values of
ALPHA and BETA

```
LDA ALPHA  
STA GAMMA  
LDA BETA  
STA ALPHA  
LDA GAMMA  
STA BETA
```

```
:  
:
```

ALPHA	WORD	8
BETA	WORD	2
TEMP	RESW	1

Sheet 1

Question 7

Write a sequence of instructions to set ALPHA equal to $4 * \text{BETA} - 9$. Assume that BETA is equal to 10

LDA	BETA
MUL	FOUR
SUB	NINE
STA	ALPHA

ALPHA	RESW	1
BETA	WORD	10
FOUR	WORD	4
NINE	WORD	9

Sheet 1

Question 1

Suppose that ALPHA is an array of 100 characters. Write a sequence of instructions for SIC to set all 100 elements of the array to letter “N”.

	LDCH	NCHAR	; Load ASCII value of 'N'
	LDX	ZERO	; X = 0
LOOP	STCH	ALPHA,X	; Store one character at ALPHA + X
	TIX	HUNDRED	; X = X + 1 and compare with 100
	JLT	LOOP	; If X < 100, continue loop
	:		
	:		
ALPHA	RESB	100	; Reserve 100 bytes (array)
NCHAR	BYTE	C'N'	; Character constant
ZERO	WORD	0	
HUNDRED	WORD	100	

Sheet 1
Question 5

Write a subroutine for SIC that will read a record into a buffer. The record may be any length from 1 to 100 bytes. The end of record is marked with a “null” character (ASCII code 00). The subroutine should place the length of the record read into a variable named LENGTH.

Subroutine Name			
READREC LOOP DONE	LDX	ZERO	; X = 0 (index / counter)
	RD	INDEV	; Read one character into A
	COMP	ZERO	; Compare with 00 (null)
	JEQ	DONE	; If null → end of record
	STCH	BUFFER,X	; Store character in buffer
	TIX	HUNDRED	; X = X + 1, compare with 100
	JLT	LOOP	; If X < 100 → continue
	STX	LENGTH	; Store number of characters read
	RSUB		; Return to caller
BUFFER	RESB	100	; Buffer for 100 bytes
LENGTH	RESW	1	; Record length
ZERO	WORD	0	
HUNDRED	WORD	100	
INDEV	BYTE	X'F1'	; Example input device code

Sheet 1

Question 6

Suppose that ALPHA and BETA are the two arrays of 10 words each (ALPHA = {4,2,4,6,8,0,3,5,9,12} and BETA={10,2,9,1,5,7,15,4,4,1}).

Write an assembly code that computes another array GAMMA whose elements are obtained by multiplying the corresponding ALPHA element by 4 and adding the corresponding BETA elements.

```
LOOP    LDX    ZERO
        LDA    ALPHA,X
        MUL    FOUR
        ADD    BETA,X
        STA    GAMMA,X
        LDA    XREG
        ADD    THREE
        STA    XREG
        LDX    XREG
        COMP   THIRTY
        JLT    LOOP
        RSUB
```

```
ZERO    WORD    0
FOUR     WORD    4
THREE    WORD    3
THIRTY   WORD    30
XREG     WORD    0
ALPHA    WORD    4,2,4,6,8,0,3,5,9,12
BETA     WORD    10,2,9,1,5,7,15,4,4,1
GAMMA    RESW    10
```